



Quest Tree Hackathon

Binary Trees — Traversals, Recursion, BST & Views · Problem Statements

DEVELOPED BY TALENCIAGLOBAL



About This Hackathon

Quest Tree is an adventurer's branching map modelled as a binary tree. Across six levels you will traverse the map, measure its height and diameter, validate and search a treasure-sorted (BST) map, check its balance with a level-order sweep, and produce a final scout's report — all built on a level-order tree description.

Input Format

Every tree is given in **level order** with **N** marking a missing child.

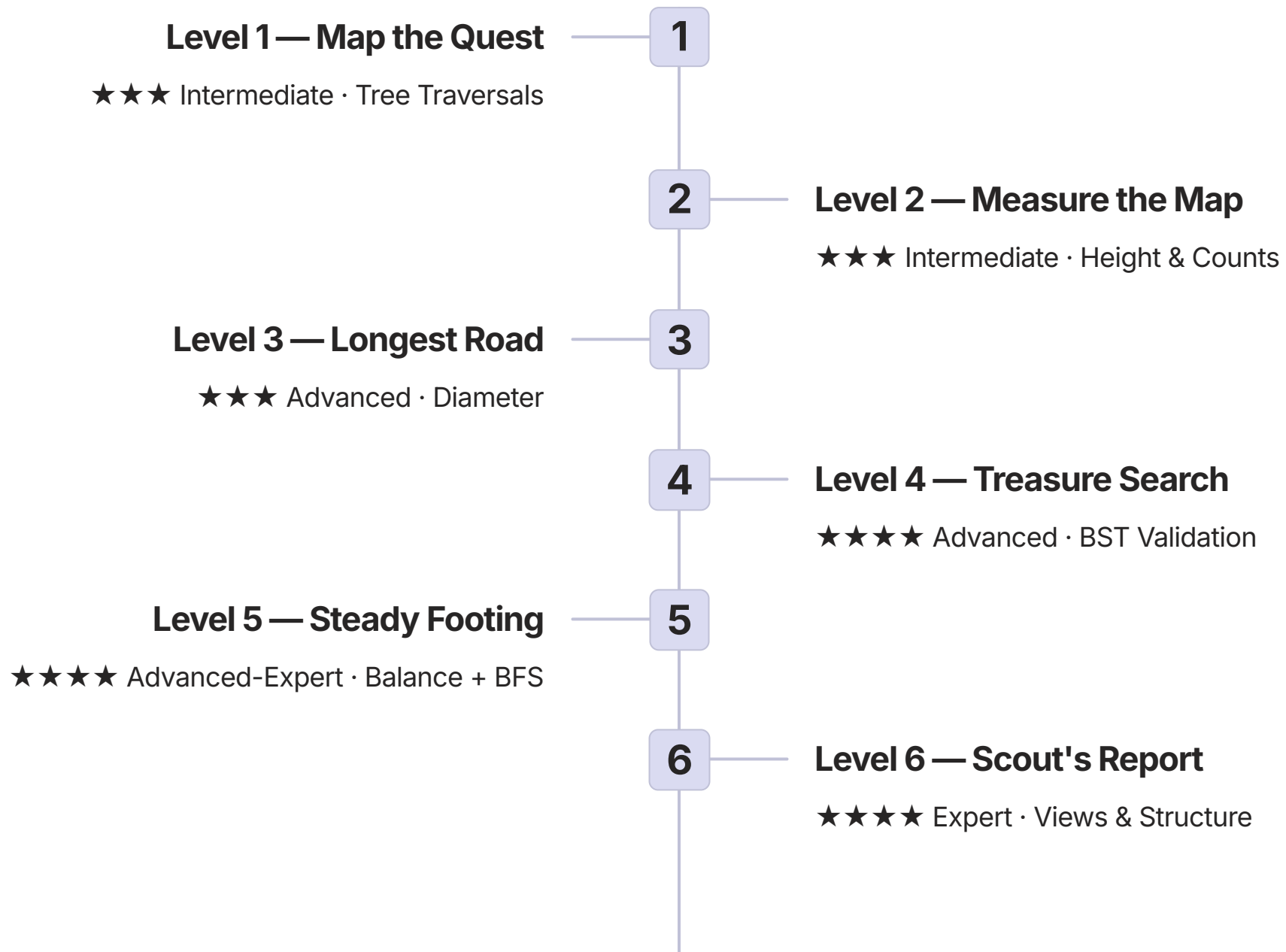
Measurement Convention

Heights and diameters are measured in **edges**. Empty tree = -1 , single node = 0 .

No Starter Code

Design and write each solution yourself from scratch.

The Six Levels at a Glance





LEVEL 1

Map the Quest

Tree Traversals · ★★ ★ Intermediate

Every Quest Tree map is a binary tree of locations. Build it from a level-order description, then walk it three ways — preorder, inorder and postorder — the three depth-first routes through any binary tree.

Level 1 — Concepts & Problem Statement

Concepts Covered

- Building a binary tree from level-order input
- **Preorder** — node, left, right
- **Inorder** — left, node, right
- **Postorder** — left, right, node

Problem Statement

Given: A binary tree in level order, with N marking a missing child.

Required output:

- The preorder traversal
- The inorder traversal
- The postorder traversal

Constraints: Node values are integers; N denotes a null child. Build the tree level by level, attaching left then right children. Each traversal is a single depth-first pass.

Level 1 — Sample Run & Key Insight

Sample Input

```
1 2 3 4 5 N 6
```

Sample Output

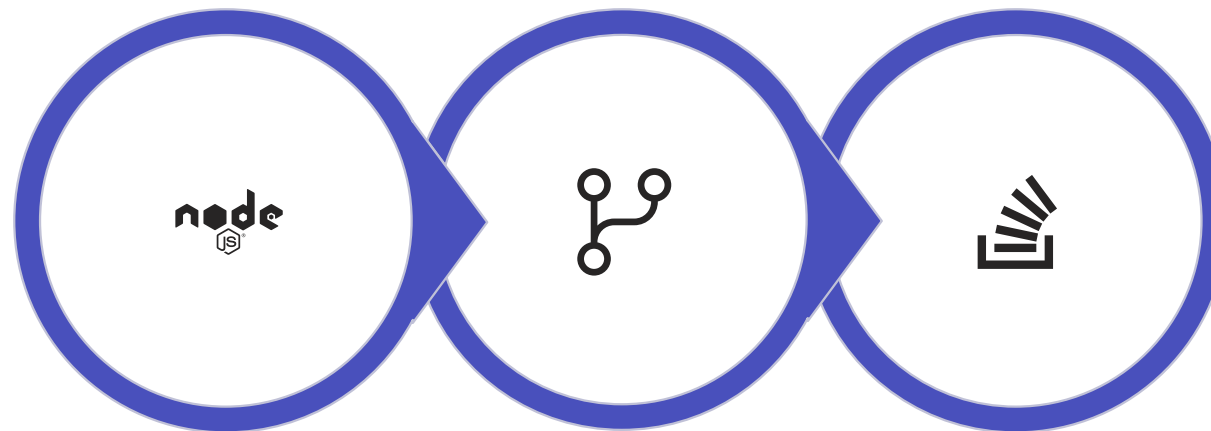
Preorder: 1 2 4 5 3 6

Inorder: 4 2 5 1 3 6

Postorder: 4 5 2 6 3 1

⚠ Common Pitfall: When building from level order, attach BOTH a left and a right slot per parent and skip N markers, or the shape (and every traversal) comes out wrong.

ℹ Thinking Skill: All three depth-first traversals are the same recursion; only the moment you record the node changes — before, between, or after the child calls.



Preorder

Inorder

Postorder



LEVEL 2

Measure the Map

Height, Counts (Recursive) · ★★★ Intermediate

Before setting out, scout the map's size: how tall it is, how many locations it holds, and how many are dead-ends (leaves). Each answer is a small recursive sweep.

Level 2 — Concepts & Formulas

Concepts Covered

Recursive Height

Measured in edges

Node Count

Total nodes in tree

Leaf Count

Nodes with no children

Subtree Combine

Merge results from left & right

Formula Guidance

```
height(n) = -1 if empty
           else 1 + max(height(left),
                        height(right))
```

```
nodes(n) = 0 if empty
          else 1 + nodes(left) + nodes(right)
```

```
leaves(n) = 1 if leaf
           else leaves(left) + leaves(right)
```



Common Pitfall: Use the edge convention consistently (empty = -1, single node = 0); mixing edges and nodes shifts every height by one.

Level 2 — Sample Run & Thinking Skill

Sample Input

```
1 2 3 4 5 N 6
```

Sample Output

```
Height: 2
```

```
Nodes: 6
```

```
Leaves: 3
```



Thinking Skill: Tree recursion follows the structure: solve the two subtrees, then combine — the base case (empty subtree) is what makes the recursion terminate and the counts correct.

Input format: One line — the tree in level order (values and N).

Output format: Three lines: Height, Nodes, Leaves.



LEVEL 3

Longest Road

Diameter · ★★ ★ Advanced

The longest road across the kingdom is the tree's diameter: the greatest number of edges on any path between two locations, even one that does not pass through the capital (the root).

Level 3 — Diameter Deep Dive

Concepts Covered

1 Diameter of a Binary Tree

Greatest number of edges on any path between two nodes.

2 Combining Heights at Each Node

Left height + right height + 2 edges gives the candidate diameter.

3 Single-Pass Recursion

One post-order pass computes both height and diameter simultaneously.

4 Root-Independent Paths

The longest path may bend at any node and need not pass through the root.

Formula Guidance

at each node:

candidate = height(left)
+ height(right) + 2

diameter = max(candidate)
over all nodes

height returns:

1 + max(left, right) as it unwinds

Sample Run

Input: 1 2 3 4 5 N 6 7 N N N N 8

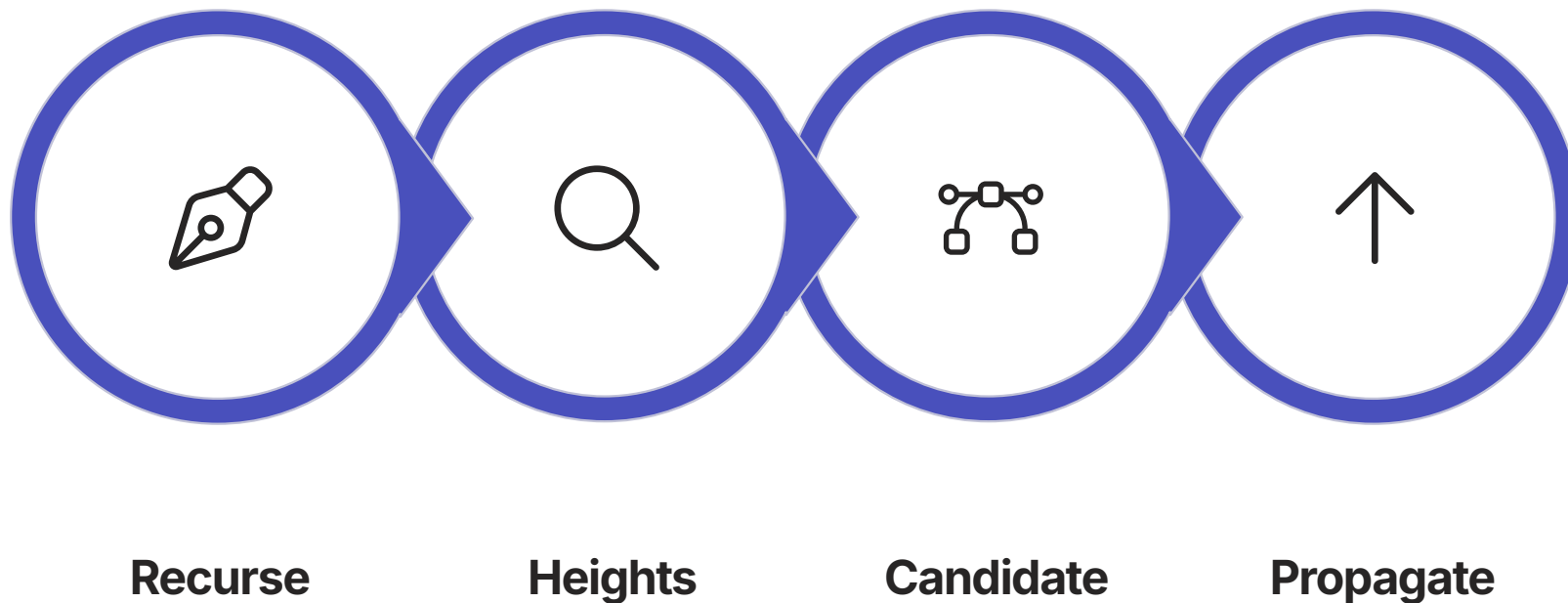
Output: Diameter: 6



Common Pitfall: Combine the two child heights at each node (not just along one branch), and remember the deepest path may not touch the root at all.

Level 3 — The Key Insight

Thinking Skill: The diameter is found by realising the longest path through any node joins that node's two deepest branches — so one height recursion, augmented to track the best join, suffices in **$O(n)$** .



This elegant single-pass approach avoids the $O(n^2)$ cost of calling a separate height function for every node, making it both correct and efficient.

LEVEL 4

Treasure Search

BST Validation, Search & Aggregation · ★★★★★ Advanced

A treasure-sorted map is a binary search tree. Confirm the map really obeys the BST rule, total its values, find the smallest and largest treasures, and check whether a specific treasure is present.



Level 4 — BST Concepts & Problem

Concepts Covered



BST Validation

Validate the BST property using inherited value ranges — not just immediate children.



Aggregation

Compute sum, min, and max over the entire tree in a single traversal.



Key Search

Search for a specific value and report Yes or No.

Problem Statement

Given: A binary tree in level order, then a search key.

Required output:

- Whether the tree is a valid BST
- The sum of all values
- The minimum and maximum values
- Whether the key is present

Constraints: A valid BST requires every left-subtree value below the node and every right-subtree value above it. The key search reports Yes or No.

Level 4 — Formula & Sample Run

Formula Guidance

```
is_bst(n, lo, hi):  
  n.v must satisfy  $lo < n.v < hi$   
  recurse with tightened bounds:  
    left  $\rightarrow$  is_bst(left, lo, n.v)  
    right  $\rightarrow$  is_bst(right, n.v, hi)
```

```
sum / min / max:  
  aggregate over in-order traversal
```



Common Pitfall: Validate against inherited bounds, not just immediate children — a left child smaller than its parent can still be larger than an ancestor and break the BST property.



Thinking Skill: BST validity is global, not local: a node can be larger than its child yet still violate an ancestor's bound, so the check must carry down a tightening (low, high) range.

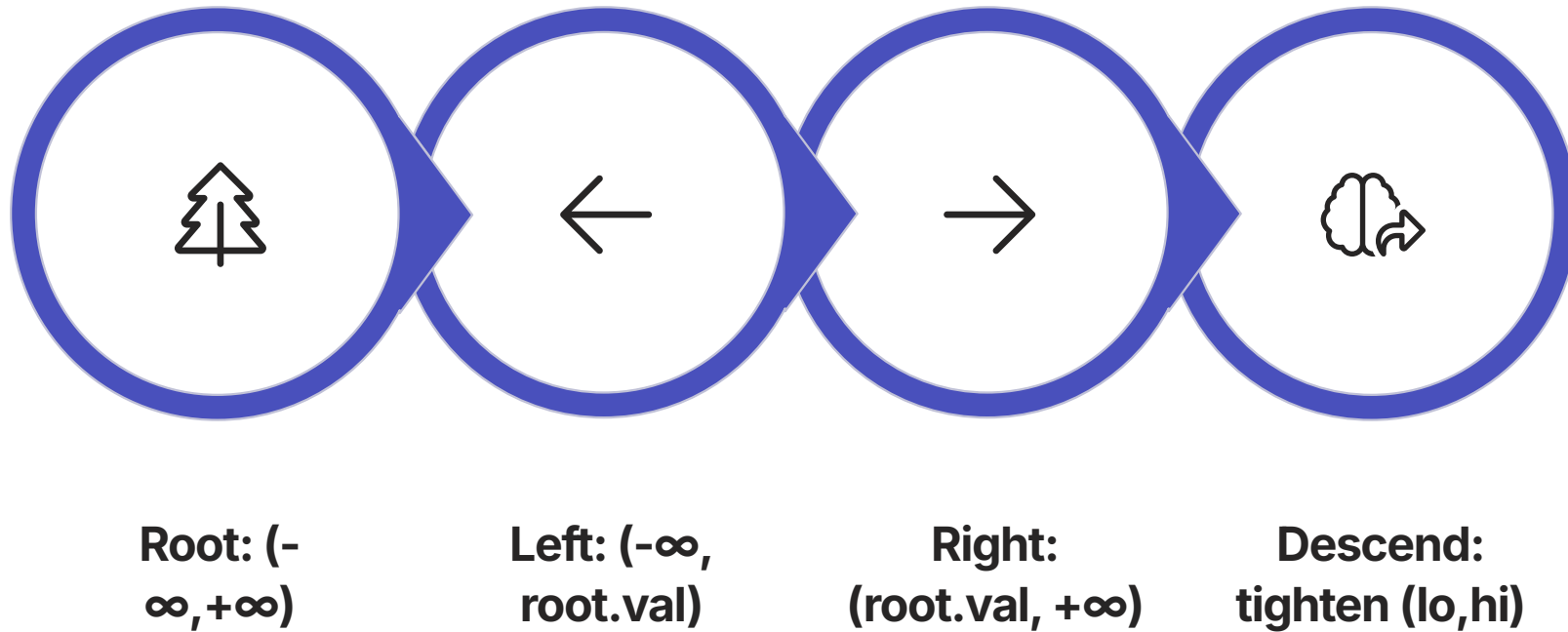
Sample Input

```
5 3 8 2 4 7 9  
7
```

Sample Output

```
Valid BST: Yes  
Sum: 38  
Min: 2  
Max: 9  
Found 7: Yes
```

BST Validation — The Range Propagation Pattern



The key insight is that each node must satisfy not just its parent's constraint, but the accumulated constraints of all its ancestors. Passing a tightening (lo, hi) range down the recursion captures this global requirement elegantly.

LEVEL 5

Steady Footing

Balance Check + Level Order (BFS) · ★★★★★ Advanced-Expert

A treacherous map is wildly lopsided. Check whether the tree is height-balanced — every node's two subtrees differ in height by at most one — and list the locations level by level with a breadth-first sweep.



Level 5 — Concepts & Problem Statement

Concepts Covered

Height-Balanced Definition

Every node's subtrees differ in height by at most 1.

Single-Pass Balance + Height

Fold the balance check into the height recursion.

Level-Order Traversal

BFS with a queue — enqueue left then right child.

BFS vs DFS

Breadth-first visits nodes top-to-bottom, left-to-right.

Formula Guidance

balance **check**:

$|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$

must **hold at EVERY** node

level **order** (BFS):

queue = [root]

while queue **not empty**:

node = dequeue

visit node

enqueue **left, then right**

Sample Run

Input: 1 2 3 4 5 N 6

Output:

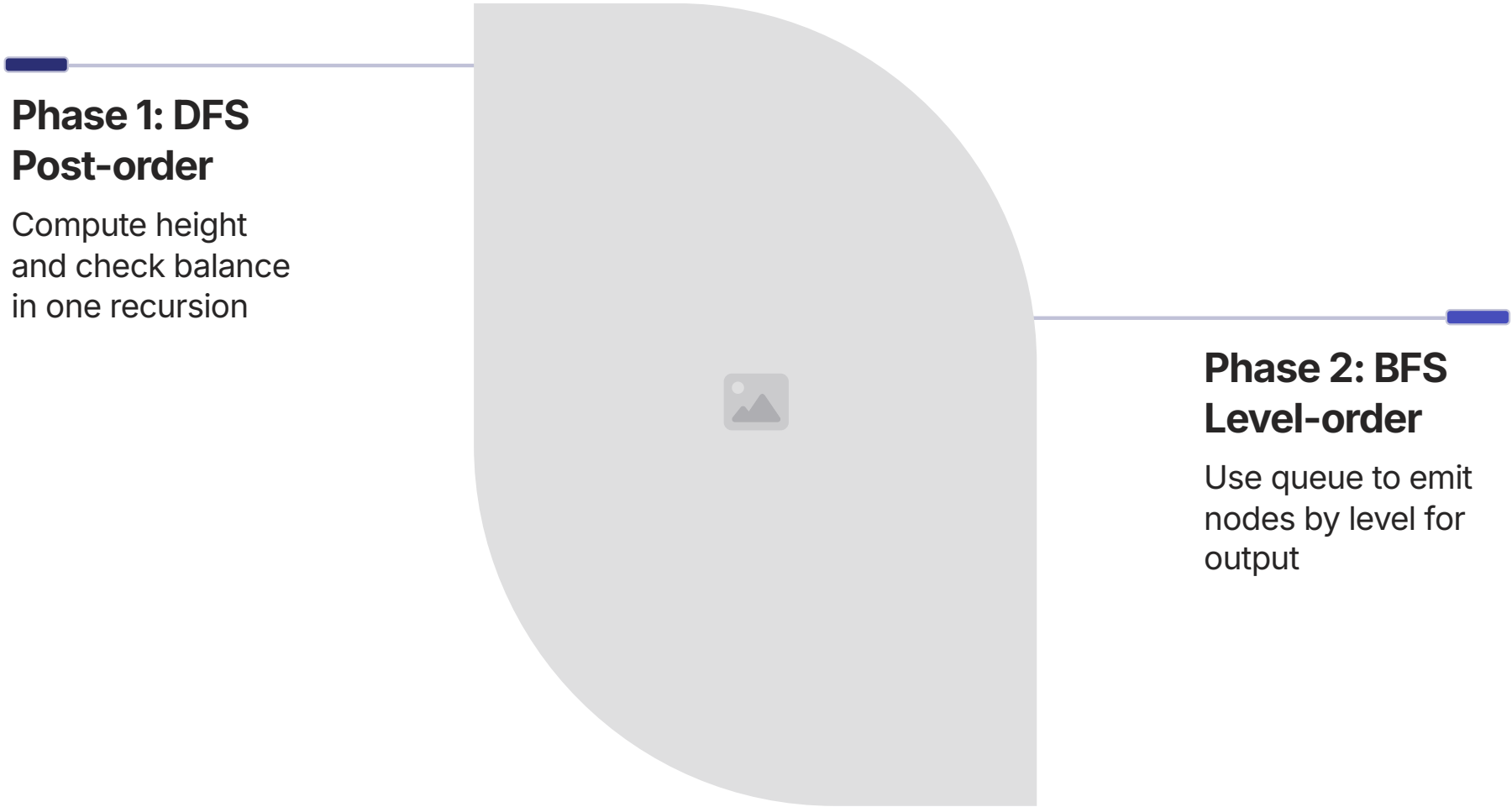
Balanced: Yes

Level Order: 1 2 3 4 5 6

Level 5 — Balance Check Insight

⚠ **Common Pitfall:** Balance must hold at EVERY node, not just the root; fold the check into the height recursion so a deep imbalance anywhere is caught.

📖 **Thinking Skill:** Checking balance in one pass works because the same recursion that measures height can compare its two child heights on the way up — no need for a separate height call per node (which would be $O(n^2)$).



Phase 1: DFS Post-order

Compute height
and check balance
in one recursion

Phase 2: BFS Level-order

Use queue to emit
nodes by level for
output

The elegance of this approach is that the balance check piggybacks on the height computation — a single post-order pass catches any imbalance anywhere in the tree, achieving $O(n)$ time complexity.



LEVEL 6

Scout's Report

Views & Structure (Capstone) · ★★★★★ Expert

The final scouting report rolls up the map's vital statistics — its height, its longest road (diameter), whether it is balanced — and the right view: the locations you would see looking at the map from the right, one per level.

Level 6 — Concepts & Problem Statement

Concepts Covered



Combining Analyses

Reuse height, diameter, and balance from earlier levels.



Right View

The last node per level — what you see from the right side.



Level-Aware BFS

Process a full level at a time to extract the rightmost node.

Problem Statement

Given: A binary tree in level order.

Required output:

- The height (in edges)
- The diameter (in edges)
- Whether the tree is balanced
- The right view (rightmost node of each level)

Input format: One line — the tree in level order (values and N).

Output format: Four lines: Height, Diameter, Balanced, Right View.

Level 6 — Formula & Sample Run

Formula Guidance

height & diameter:
post-order height recursion
track $\max(\text{left_h} + \text{right_h} + 2)$

balance:
same recursion, check
 $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$
at every node

right view:
BFS, process full level at a time
take the LAST node dequeued per level

Sample Input

1 2 3 N 4 N 5

Sample Output

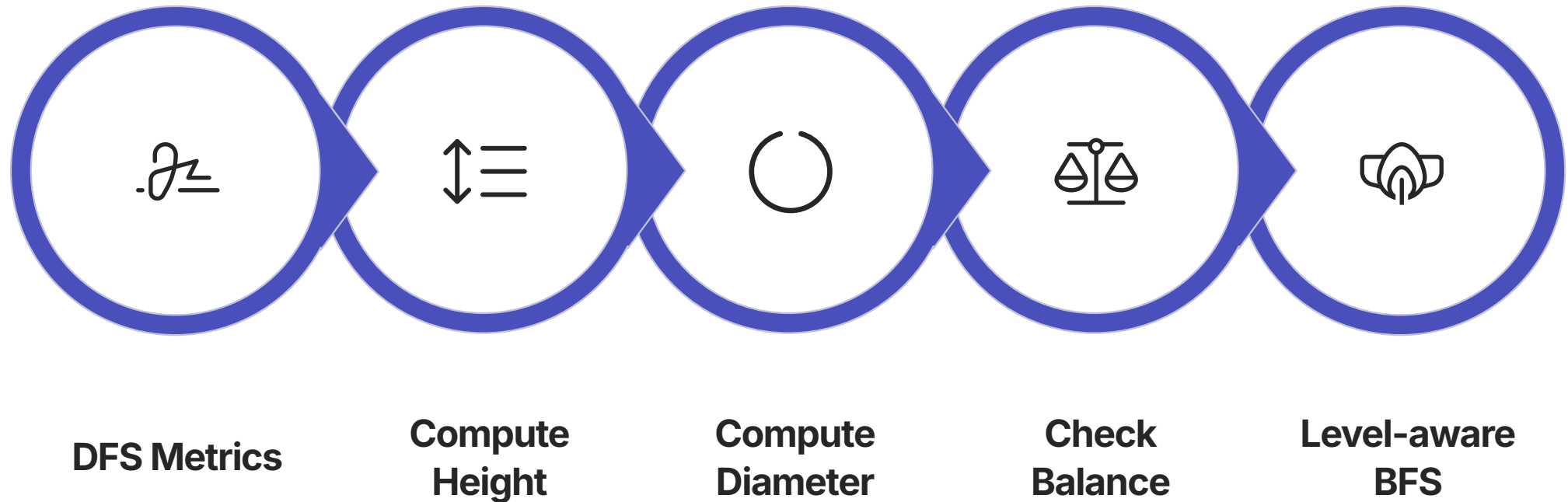
Height: 2
Diameter: 4
Balanced: Yes
Right View: 1 3 5



Common Pitfall: For the right view, take the LAST node dequeued on each level (process a full level at a time); taking the first gives the left view instead.

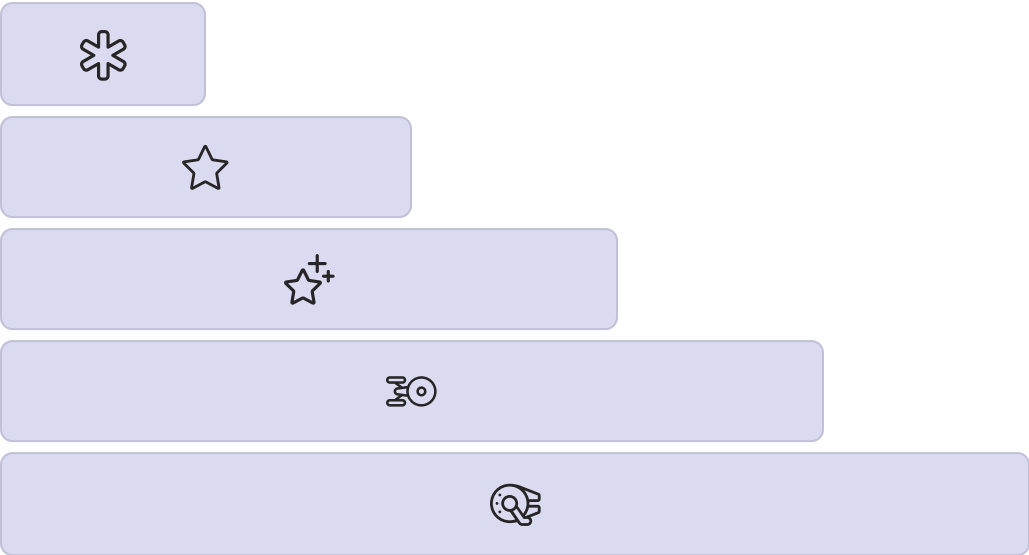
Level 6 — The Capstone Insight

Thinking Skill: A capstone tree report is just the earlier passes composed — depth-first recursions for the metrics, a level-aware breadth-first sweep for the view — the same tree, examined from several angles.



Level 6 is not a new algorithm — it is the art of composition. Every technique from Levels 1–5 is reused and combined into a single coherent report, demonstrating that mastery of tree algorithms means knowing which tool to reach for and how to combine them efficiently.

Difficulty Progression



Levels 1–2: Intermediate

Build the tree, walk it three ways, measure height and count nodes recursively.



Level 3: Advanced

Diameter — the longest path, found in a single augmented post-order pass.



Level 4: Advanced

BST validation with global range propagation, aggregation, and search.



Level 5: Advanced-Expert

Balance check folded into height recursion; BFS level-order traversal.



Level 6: Expert

Capstone — compose all prior techniques plus the right-view BFS sweep.

Core Algorithmic Themes

Recursion

Every metric — height, count, diameter, balance — is a recursive post-order computation.

BFS with a Queue

Level-order traversal and right-view extraction both rely on breadth-first search.

Global Constraints

BST validation requires carrying inherited bounds — local checks are not enough.

Single-Pass Efficiency

Combining multiple computations in one pass achieves $O(n)$ across all levels.

Ready to Begin Your Quest?

Six levels. Six algorithms. One tree. From building and traversing in Level 1 to composing a full scout's report in Level 6 — every challenge builds on the last. Master the recursion, wield the queue, and conquer the Quest Tree.

Remember the Convention

Empty tree = -1 edges. Single node = 0 edges. Stay consistent throughout.

One Pass Is Enough

Height, diameter, and balance can all be computed in a single post-order recursion.

Global, Not Local

BST validation and balance checks must hold at every node — not just the root.

Compose & Conquer

The capstone is just the earlier passes combined — reuse what you've already built.

DEVELOPED BY [TALENCIAGLOBAL](#)

